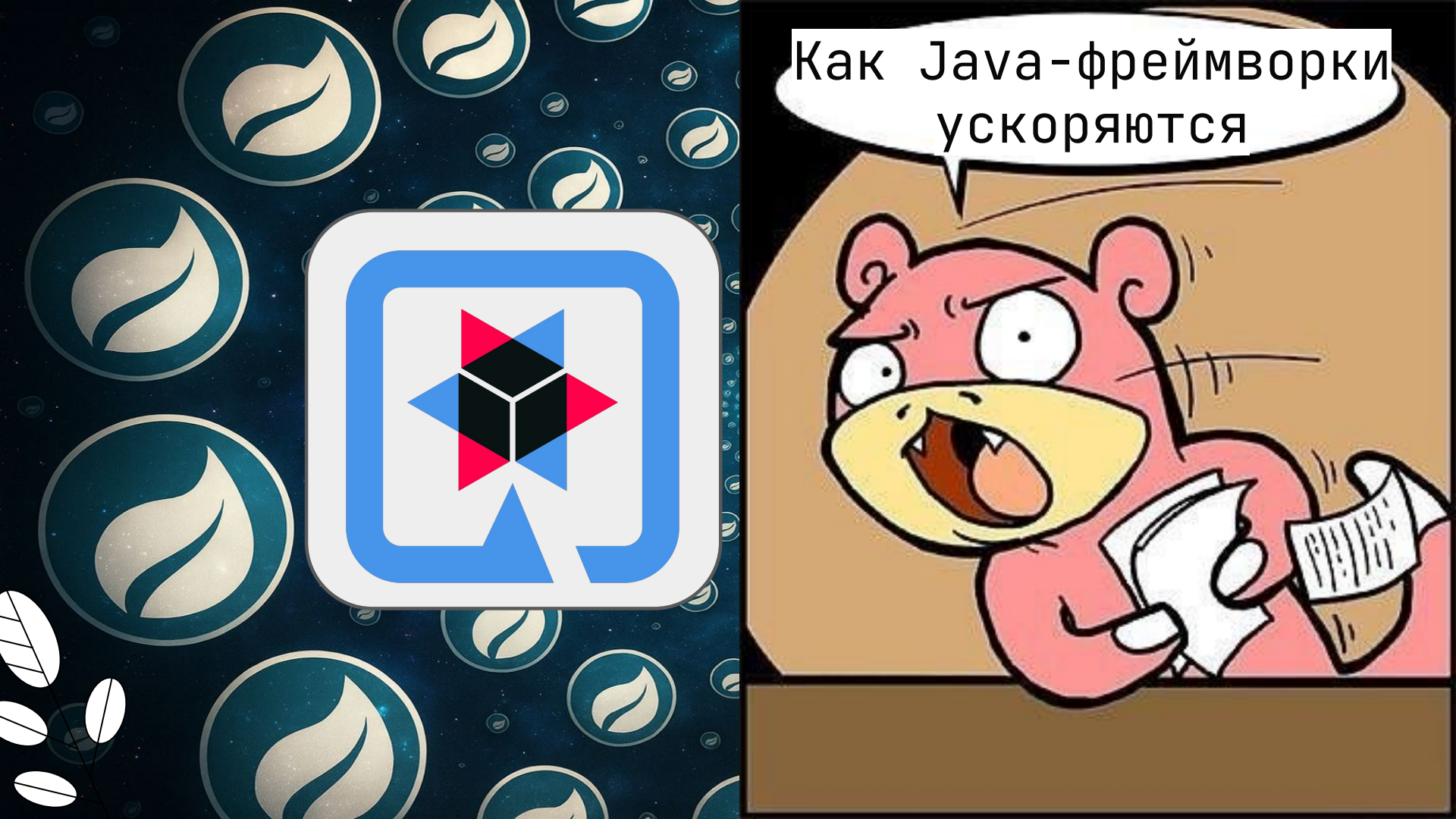

Как Java-фреймворки ускоряются

# $ whoami?



JPoint

Joker<?>

Code Fest

sysconf

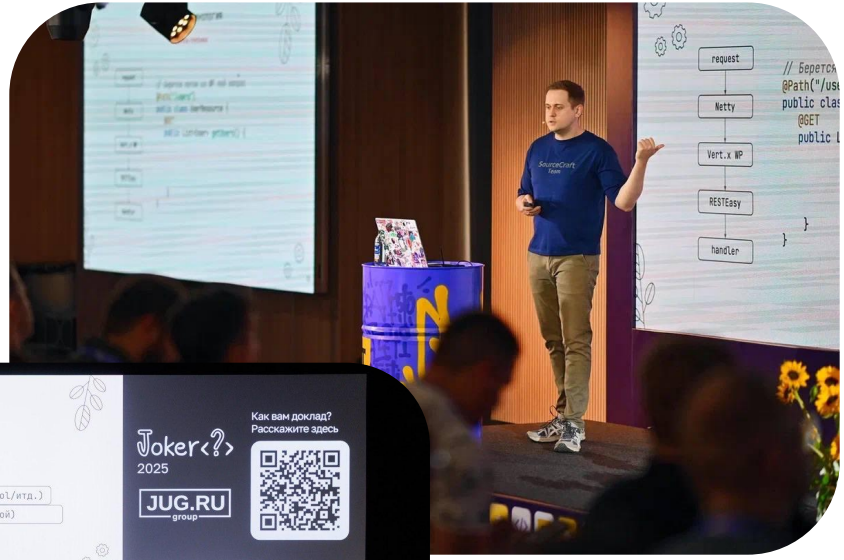
podlodka

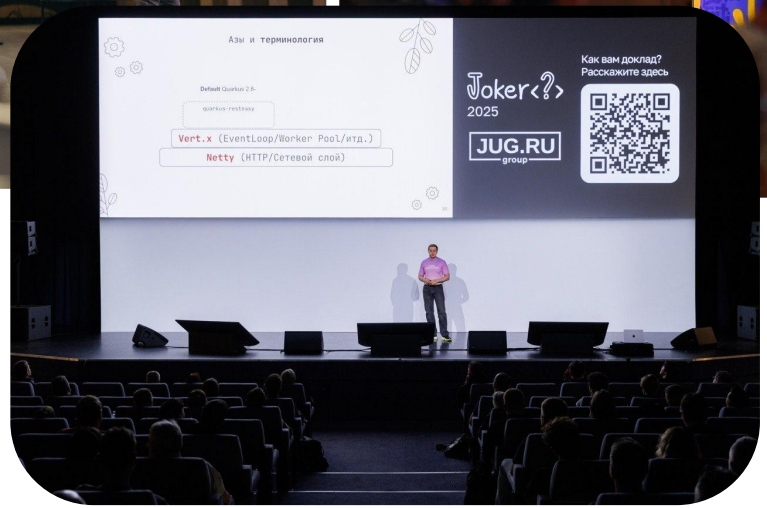
Андрей Кулешов

intel. Deutsche Bank HUAWEI pt Я

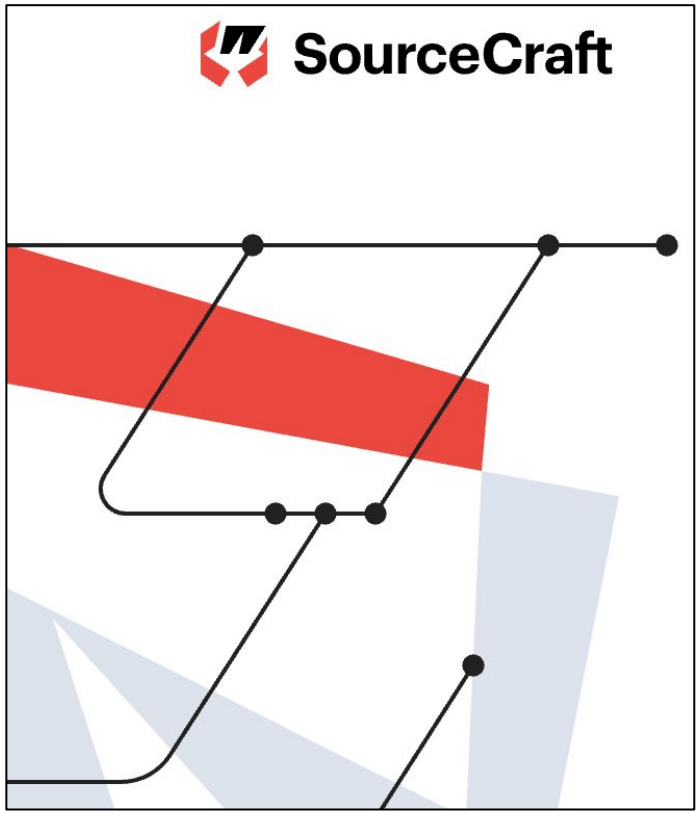
# В предыдущих сериях





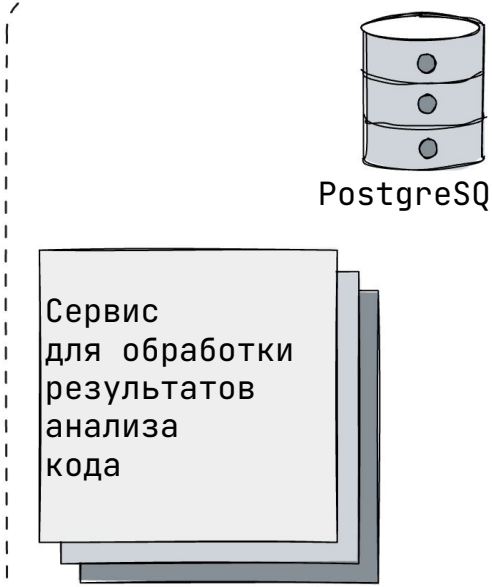



# Моя исто|

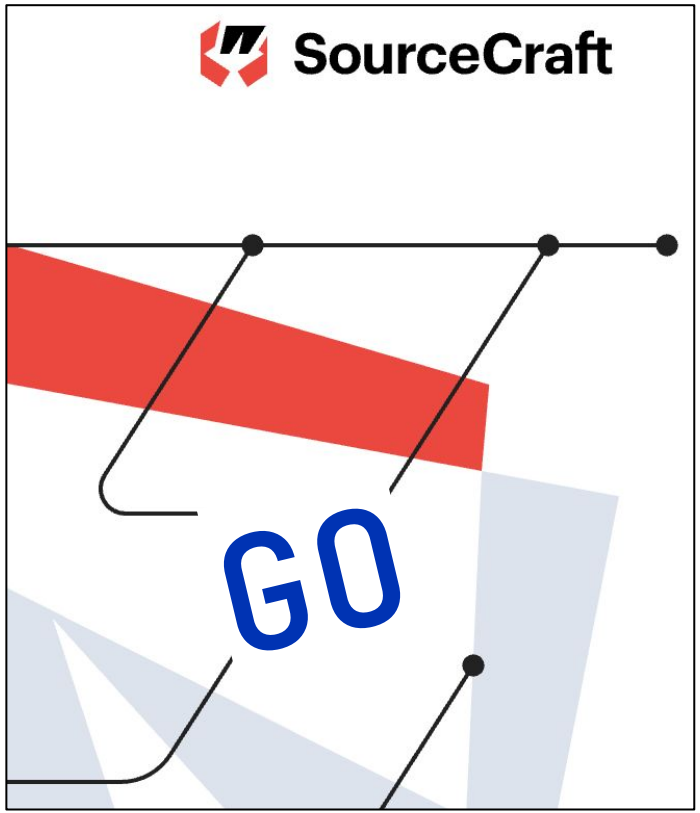



gRPC

REST

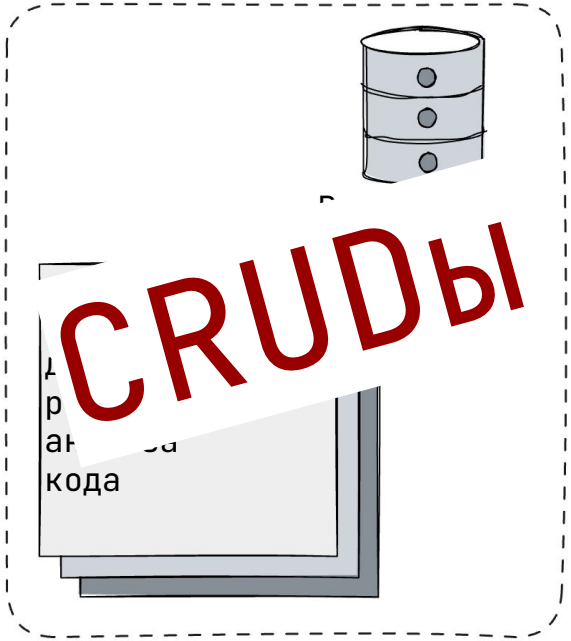


# Моя история




gRPC

REST

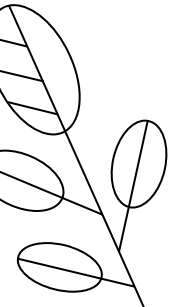


# Вопрос чести жависта!



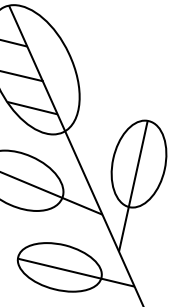
# По итогу

- Год крутим небольшой сервис на Quarkus JVM в PROD
- 50k строк, 60 grpc, десяток REST эндпоинтов
- 0 инцидентов на стороне фреймворка
- Честный старт (не lazy) за ~3 секунды

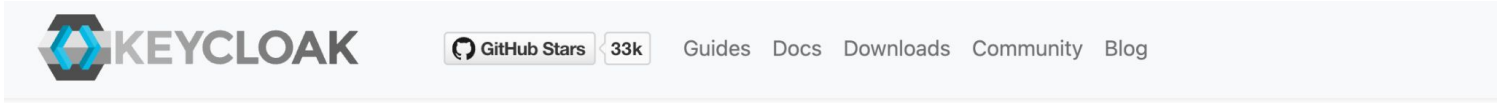


# По итогу

Да и вы пользовались кваркусом



# По итогу




Guides / Migration / Migrating to Quarkus distribution

## Migrating to Quarkus distribution

26.5.5

Migrate to the new Quarkus distribution from the legacy WildFly distribution

Starting with Keycloak 17 the default distribution is now powered by Quarkus, while the legacy WildFly powered distribution will still be around until June 2022 we highly recommend starting the migration as soon as possible.

The new distribution introduces a number of breaking changes, including:

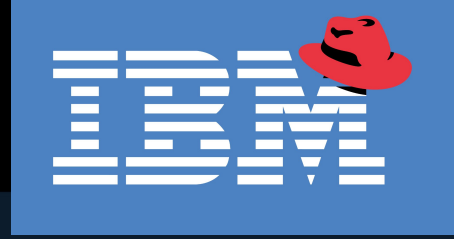
- Configuring Keycloak has significantly changed
- Quarkus is not an application server, but rather a framework to build applications
- `/auth` removed from the default context path
- Custom providers are packaged and deployed differently
- A new operator and CRDs for Kubernetes and OpenShift

Before undertaking the migration we highly recommend reading through the new Server Guides to understand how to install and configure the new distribution.


How it started
How it’s going

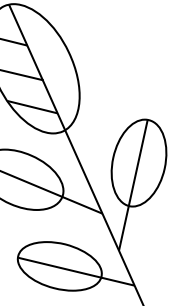
QUARKUS

WHY LEARN EXTENSIONS COMMUNITY START CODING



# Quarkus Performance

## Designed for Fast Startup, High Throughput, and Low Resource Consumption





Quarkus is engineered to be efficient by using build-time optimizations and a reactive core to achieve fast startup times, high throughput, low response latency, reduced memory footprint, and minimal resource consumption. As a result, Quarkus is fast... real fast.

QUARKUS

WHY LEARN EXTENSIONS COMMUNITY START CODING

# Quarkus Performance

Designed for Fast Startup, High Throughput, and Low Resource Consumption



Quarkus is engineered to be efficient by using build-time optimizations and a reactive core to achieve fast startup times, high throughput, low response latency, reduced memory footprint, and minimal resource consumption. As a result, Quarkus is fast... real fast.

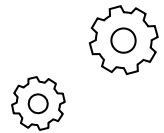
# Continuously Measuring, Continuously Improving

Quarkus is dedicated to continuously improving performance, especially for code running on the critical execution (hot) path. Through ongoing optimizations, Quarkus ensures that every instruction and allocated byte matters, making it one of the most efficient frameworks available for developing high-performance, cloud-ready applications.



Куда ведет ссылка?





# Continuously Measuring, Continuously Improving

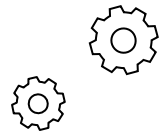
Quarkus is dedicated to continuously improving performance, especially for code running on the critical execution (hot) path. Through ongoing optimizations, Quarkus ensures that every instruction and allocated byte matters, making it one of the most efficient frameworks available for developing high-performance, cloud-ready applications.



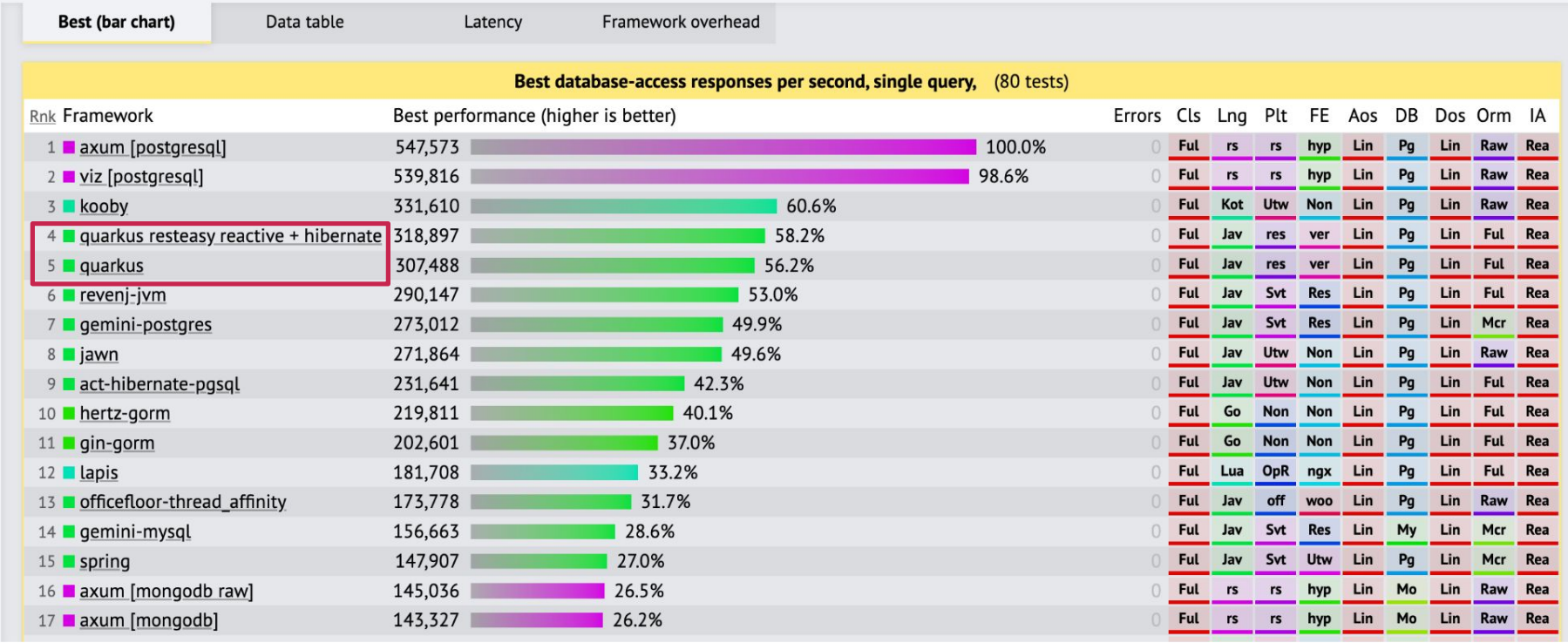
Куда ведет ссылка?

Правильно, не лучший TechEmpower





# Single query



Best (bar chart) | Data table | Latency | Framework overhead

**Best database-access responses per second, single query,** (80 tests)

| Rnk | Framework | Best performance (higher is better) | | Errors | Cls | Lng | Plt | FE | Aos | DB | Dos | Orm | IA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | axum [postgresql] | 547,573 | 100.0% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 2 | viz [postgresql] | 539,816 | 98.6% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 3 | kooby | 331,610 | 60.6% | 0 | Ful | Kot | Utw | Non | Lin | Pg | Lin | Raw | Rea |
| 4 | quarkus resteasy reactive + hibernate | 318,897 | 58.2% | 0 | Ful | Jav | res | ver | Lin | Pg | Lin | Ful | Rea |
| 5 | quarkus | 307,488 | 56.2% | 0 | Ful | Jav | res | ver | Lin | Pg | Lin | Ful | Rea |
| 6 | revenj-jvm | 290,147 | 53.0% | 0 | Ful | Jav | Svt | Res | Lin | Pg | Lin | Ful | Rea |
| 7 | gemini-postgres | 273,012 | 49.9% | 0 | Ful | Jav | Svt | Res | Lin | Pg | Lin | Mcr | Rea |
| 8 | jawn | 271,864 | 49.6% | 0 | Ful | Jav | Utw | Non | Lin | Pg | Lin | Raw | Rea |
| 9 | act-hibernate-pgsql | 231,641 | 42.3% | 0 | Ful | Jav | Utw | Non | Lin | Pg | Lin | Ful | Rea |
| 10 | hertz-gorm | 219,811 | 40.1% | 0 | Ful | Go | Non | Non | Lin | Pg | Lin | Ful | Rea |
| 11 | gin-gorm | 202,601 | 37.0% | 0 | Ful | Go | Non | Non | Lin | Pg | Lin | Ful | Rea |
| 12 | lapis | 181,708 | 33.2% | 0 | Ful | Lua | OpR | ngx | Lin | Pg | Lin | Ful | Rea |
| 13 | officefloor-thread_affinity | 173,778 | 31.7% | 0 | Ful | Jav | off | woo | Lin | Pg | Lin | Raw | Rea |
| 14 | gemini-mysql | 156,663 | 28.6% | 0 | Ful | Jav | Svt | Res | Lin | My | Lin | Mcr | Rea |
| 15 | spring | 147,907 | 27.0% | 0 | Ful | Jav | Svt | Utw | Lin | Pg | Lin | Mcr | Rea |
| 16 | axum [mongodb raw] | 145,036 | 26.5% | 0 | Ful | rs | rs | hyp | Lin | Mo | Lin | Raw | Rea |
| 17 | axum [mongodb] | 143,327 | 26.2% | 0 | Ful | rs | rs | hyp | Lin | Mo | Lin | Raw | Rea |

JSON serialization | Single query | **Multiple queries** | Cached queries | Fortunes | Data updates | Plaintext

# Multiple queries

**20-queries (bar)** | Data table | Latency | Framework overhead

**Responses per second at 20 queries per request,** (542 tests)

| Rnk | Framework | Performance (higher is better) | | Errors | Cls | Lng | Plt | FE | Aos | DB | Dos | Orm | IA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | just-js | 89,470 | 100.0% | 0 | Plt | JS | jus | Non | Lin | Pg | Lin | Raw | Rea |
| 2 | vertx-web-kotlinx-postgresql | 89,289 | 99.8% | 0 | Mcr | Kot | ver | Non | Lin | Pg | Lin | Raw | Rea |
| 3 | salvo [postgres] | 89,194 | 99.7% | 0 | Mcr | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 4 | quarkus, vert.x | 88,902 | 99.4% | 0 | Mcr | Jav | ver | ver | Lin | Pg | Lin | Raw | Rea |
| 5 | vertx-postgres | 88,708 | 99.1% | 0 | Plt | Jav | ver | Non | Lin | Pg | Lin | Raw | Rea |
| 6 | ntex [sailfish] | 88,589 | 99.0% | 0 | Mcr | rs | Non | nte | Lin | Pg | Lin | Raw | Rea |
| 7 | xitca-web | 88,534 | 99.0% | 0 | Mcr | rs | Non | xit | Lin | Pg | Lin | Raw | Rea |
| 8 | viz [postgresql] | 88,468 | 98.9% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 9 | xitca-web [orm] | 88,296 | 98.7% | 0 | Ful | rs | Non | xit | Lin | Pg | Lin | Ful | Rea |
| 10 | ntex [async-std,db] | 88,149 | 98.5% | 0 | Mcr | rs | Non | nte | Lin | Pg | Lin | Raw | Rea |
| 11 | may-minihttp | 88,108 | 98.5% | 0 | Mcr | rs | rs | may | Lin | Pg | Lin | Raw | Rea |
| 12 | inverno | 87,919 | 98.3% | 0 | Mcr | Jav | inv | Non | Lin | Pg | Lin | Mcr | Rea |
| 13 | axum [postgresql] | 87,231 | 97.5% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 14 | h2o | 86,788 | 97.0% | 0 | Plt | C | Non | h2o | Lin | Pg | Lin | Raw | Rea |

# Fortunes – лучшая производительность по количеству ответов в секунду

JSON serialization | Single query | Multiple queries | Cached queries | **Fortunes** | Data updates | Plaintext

## Fortunes

**Best fortunes responses per second,** (523 tests)

| Rnk | Framework | Best performance (higher is better) | | Errors | Cls | Lng | Plt | FE | Aos | DB | Dos | Orm | IA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | may-minihttp | 1,327,378 | 100.0% | 0 | Mcr | rs | rs | may | Lin | Pg | Lin | Raw | Rea |
| 2 | h2o | 1,226,814 | 92.4% | 0 | Plt | C | Non | h2o | Lin | Pg | Lin | Raw | Rea |
| 3 | ntex [sailfish] | 1,210,348 | 91.2% | 0 | Mcr | rs | Non | nte | Lin | Pg | Lin | Raw | Rea |
| 4 | ntex [async-std,db] | 1,197,351 | 90.2% | 0 | Mcr | rs | Non | nte | Lin | Pg | Lin | Raw | Rea |
| 5 | xitca-web | 1,146,712 | 86.4% | 0 | Mcr | rs | Non | xit | Lin | Pg | Lin | Raw | Rea |
| 6 | xitca-web [orm] | 1,115,124 | 84.0% | 0 | Ful | rs | Non | xit | Lin | Pg | Lin | Ful | Rea |
| 7 | axum [postgresql] | 1,114,265 | 83.9% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 8 | lithium-postgres | 1,073,846 | 80.9% | 0 | Mcr | C++ | Non | Non | Lin | Pg | Lin | Ful | Rea |
| 9 | lithium-postgres-beta | 1,068,560 | 80.5% | 0 | Mcr | C++ | Non | Non | Lin | Pg | Lin | Ful | Rea |
| 10 | hyper-db | 1,066,644 | 80.4% | 0 | Mcr | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 11 | viz [postgresql] | 1,060,105 | 79.9% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 12 | drogon-core | 1,042,653 | 78.5% | 0 | Ful | C++ | Non | Non | Lin | Pg | Lin | Raw | Rea |
| 13 | vertx-postgres | 1,040,599 | 78.4% | 0 | Plt | Jav | ver | Non | Lin | Pg | Lin | Raw | Rea |
| 14 | quarkus, vert.x | 1,028,408 | 77.5% | 0 | Mcr | Jav | ver | ver | Lin | Pg | Lin | Raw | Rea |

# Нативные языки

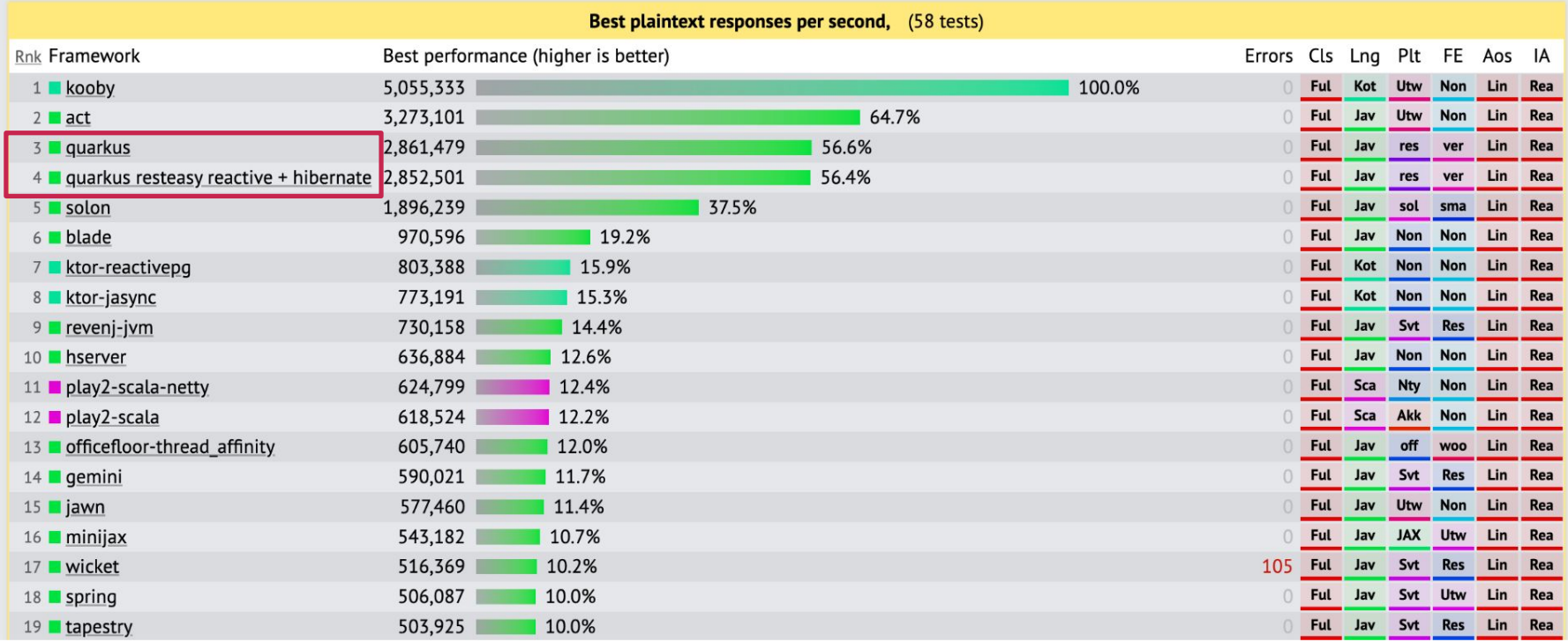
JSON serialization | Single query | Multiple queries | Cached queries | **Fortunes** | Data updates | Plaintext

## Fortunes

**Best fortunes responses per second,** (523 tests)

| Rnk | Framework | Best performance (higher is better) | | Errors | Cls | Lng | Plt | FE | Aos | DB | Dos | Orm | IA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | may-minihttp | 1,327,378 | 100.0% | 0 | Mcr | rs | rs | may | Lin | Pg | Lin | Raw | Rea |
| 2 | h2o | 1,226,814 | 92.4% | 0 | Plt | C | Non | h2o | Lin | Pg | Lin | Raw | Rea |
| 3 | ntex [sailfish] | 1,210,348 | 91.2% | 0 | Mcr | rs | Non | nte | Lin | Pg | Lin | Raw | Rea |
| 4 | ntex [async-std,db] | 1,197,351 | 90.2% | 0 | Mcr | rs | Non | nte | Lin | Pg | Lin | Raw | Rea |
| 5 | xitca-web | 1,146,712 | 86.4% | 0 | Mcr | rs | Non | xit | Lin | Pg | Lin | Raw | Rea |
| 6 | xitca-web [orm] | 1,115,124 | 84.0% | 0 | Ful | rs | Non | xit | Lin | Pg | Lin | Ful | Rea |
| 7 | axum [postgresql] | 1,114,265 | 83.9% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 8 | lithium-postgres | 1,073,846 | 80.9% | 0 | Mcr | C++ | Non | Non | Lin | Pg | Lin | Ful | Rea |
| 9 | lithium-postgres-beta | 1,068,560 | 80.5% | 0 | Mcr | C++ | Non | Non | Lin | Pg | Lin | Ful | Rea |
| 10 | hyper-db | 1,066,644 | 80.4% | 0 | Mcr | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 11 | viz [postgresql] | 1,060,105 | 79.9% | 0 | Ful | rs | rs | hyp | Lin | Pg | Lin | Raw | Rea |
| 12 | drogon-core | 1,042,653 | 78.5% | 0 | Ful | C++ | Non | Non | Lin | Pg | Lin | Raw | Rea |
| 13 | vertx-postgres | 1,040,599 | 78.4% | 0 | Plt | Jav | ver | Non | Lin | Pg | Lin | Raw | Rea |
| 14 | quarkus, vert.x | 1,028,408 | 77.5% | 0 | Mcr | Jav | ver | ver | Lin | Pg | Lin | Raw | Rea |

# Plaintext



**Best plaintext responses per second,** (58 tests)

| Rnk | Framework | Best performance (higher is better) | | Errors | Cls | Lng | Plt | FE | Aos | IA |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | kooby | 5,055,333 | 100.0% | 0 | Ful | Kot | Utw | Non | Lin | Rea |
| 2 | act | 3,273,101 | 64.7% | 0 | Ful | Jav | Utw | Non | Lin | Rea |
| 3 | quarkus | 2,861,479 | 56.6% | 0 | Ful | Jav | res | ver | Lin | Rea |
| 4 | quarkus resteasy reactive + hibernate | 2,852,501 | 56.4% | 0 | Ful | Jav | res | ver | Lin | Rea |
| 5 | solon | 1,896,239 | 37.5% | 0 | Ful | Jav | sol | sma | Lin | Rea |
| 6 | blade | 970,596 | 19.2% | 0 | Ful | Jav | Non | Non | Lin | Rea |
| 7 | ktor-reactivepg | 803,388 | 15.9% | 0 | Ful | Kot | Non | Non | Lin | Rea |
| 8 | ktor-jasync | 773,191 | 15.3% | 0 | Ful | Kot | Non | Non | Lin | Rea |
| 9 | revenj-jvm | 730,158 | 14.4% | 0 | Ful | Jav | Svt | Res | Lin | Rea |
| 10 | hserver | 636,884 | 12.6% | 0 | Ful | Jav | Non | Non | Lin | Rea |
| 11 | play2-scala-netty | 624,799 | 12.4% | 0 | Ful | Sca | Nty | Non | Lin | Rea |
| 12 | play2-scala | 618,524 | 12.2% | 0 | Ful | Sca | Akk | Non | Lin | Rea |
| 13 | officefloor-thread_affinity | 605,740 | 12.0% | 0 | Ful | Jav | off | woo | Lin | Rea |
| 14 | gemini | 590,021 | 11.7% | 0 | Ful | Jav | Svt | Res | Lin | Rea |
| 15 | jawn | 577,460 | 11.4% | 0 | Ful | Jav | Utw | Non | Lin | Rea |
| 16 | minijax | 543,182 | 10.7% | 0 | Ful | Jav | JAX | Utw | Lin | Rea |
| 17 | wicket | 516,369 | 10.2% | 105 | Ful | Jav | Svt | Res | Lin | Rea |
| 18 | spring | 506,087 | 10.0% | 0 | Ful | Jav | Svt | Utw | Lin | Rea |
| 19 | tapestry | 503,925 | 10.0% | 0 | Ful | Jav | Svt | Res | Lin | Rea |

# Утверждается

- Fast startup times

# Утверждается

- Fast startup times
- High throughput

# Утверждается

- Fast startup times
- High throughput
- Low response latency

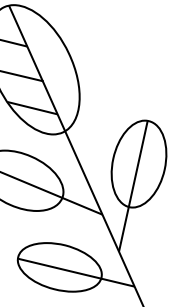
# Утверждается

- Fast startup times
- High throughput
- Low response latency
- Reduced memory footprint and minimal resource consumption

# Утверждается

- **Fast startup times**
- High throughput
- Low response latency
- Reduced memory footprint and minimal resource consumption

# Быстрый старт






Build faster, smaller, leaner applications


GraalVM™
Build faster, smaller, leaner applications

# GraalVM уходит?



**thomaswue** • 3 мес. назад

Hi, GraalVM project lead at Oracle here. Don't worry, we are not going anywhere and we will continue our mission to run programs faster and more efficient :).

82 Ответить Награда Поделиться ...



**dmigowski** • 3 мес. назад

The blog post sounds like GraalVM will vanish from the OpenJDKs and that you have to be a paying customer of Oracle to continue using it with newer JDK releases.

Could you clarify?

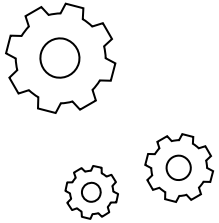
EDIT: This is from r/quarkus:

---

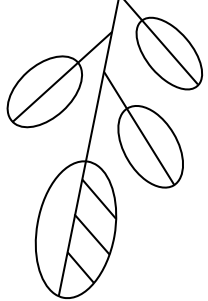
It's not dead at all, I was reassured of that. In fact, it looks like it has been promoted.

# Starting fast by doing less

- Quarkus переосмысливает жизненный цикл Java-приложений

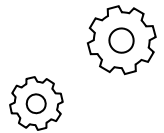


# Starting fast by doing less



- Quarkus переосмысливает жизненный цикл Java-приложений
- Максимум работы переносится с runtime на build-time





# Starting fast by doing less

- Quarkus переосмысливает жизненный цикл Java-приложений
- Максимум работы переносится с runtime на build-time
- Дорогие операции выполняются один раз при сборке, а не при каждом старте

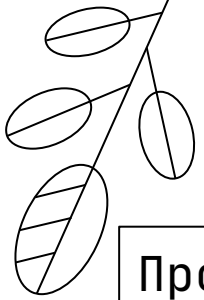
# Starting fast by doing less

- Quarkus переосмысливает жизненный цикл Java-приложений
- Максимум работы переносится с runtime на build-time
- Дорогие операции выполняются один раз при сборке, а не при каждом старте
- Результат: быстрый старт, компактность и эффективность




# Starting fast by doing less?

Как они замеряют перформанс?





## Проблема ленивой инициализации

- Приложение может:
  - быстро запуститься
  - но долго обрабатывать первый запрос

- Часть инициализации происходит:
  - при первом HTTP-запросе
  - при первом обращении к CDI / REST / сериализации

```java
@Path("/")
public class GreetingEndpoint {


    private static final String template = "Hello, %s!";


    @GET
    @Path("/greeting")
    @Produces(MediaType.APPLICATION_JSON)
    public Greeting greeting(@QueryParam("name") String name) {
         System.out.println(
                  new SimpleDateFormat("HH:mm:ss.SSS")
                           .format(new Date())
         );
        String suffix = name != null ? name : "World";
        return new Greeting(String.format(template, suffix));
    }

}
```

В приложении фиксируются:

- момент обработки первого HTTP-запроса

```java
@Path("/")
public class GreetingEndpoint {


    private static final String template = "Hello, %s!";



    void onStart(@Observes StartupEvent startup) {
         System.out.println(
                  new SimpleDateFormat("HH:mm:ss.SSS")
                           .format(new Date())
         );
    }
}
```

В приложении фиксируются:

- момент старта приложения (StartupEvent)

```
$ while [[ "$(curl -s -o /dev/null -w ''%{http_code}'' localhost:8080/api/greeting)" != "200" ]]; do sleep .00001; done
```

```
$ date +"%T.%3N" &&  ./target/quarkus-timing-runner

10:57:32.508
10:57:32.512
2019-04-05 10:57:32,512 INFO  [io.quarkus] (main) Quarkus 0.11.0 started in 0.002s. Listening on: http://127.0.0.1:8080
2019-04-05 10:57:32,512 INFO  [io.quarkus] (main) Installed features: [cdi, rest, rest-jackson]
10:57:32.537
```

Это позволяет увидеть:

- когда приложение стартовало
- когда оно реально начало обслуживать запросы

# Классический подход vs Quarkus

Обычное Java-приложение

- Сканирование классов **при старте**
- Работа с **рефлексией**
- Загрузка конфигурации **на runtime**
- Долгий startup и лишние аллокации

# Классический подход vs Quarkus

### Обычное Java-приложение

- Сканирование классов при старте
- Работа с рефлексией
- Загрузка конфигурации на runtime
- Долгий startup и лишние аллокации

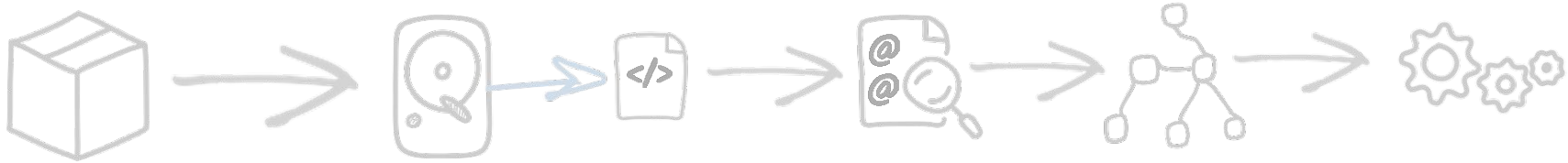
### Quarkus

- Анализ и подготовка на **этапе сборки**
- Минимум логики **при запуске**
- **Предсказуемый** и быстрый старт

## Traditional Frameworks

Build Time

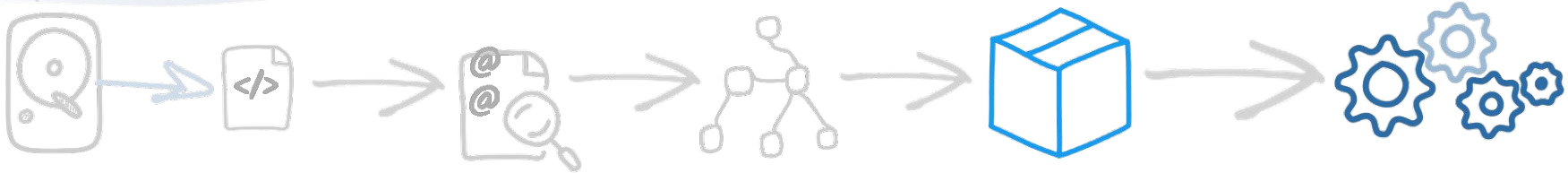
Runtime



## Quarkus

Build Time

Runtime



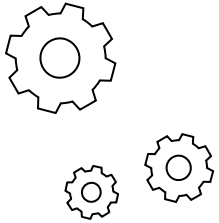
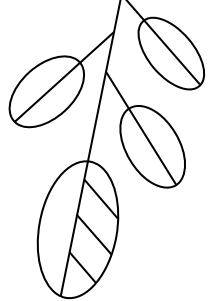
# Что Quarkus делает на этапе сборки

- Читает часть конфигурации приложения
- Сканирует classpath и аннотации
- Строит модель приложения
- Генерация кода и вмешательство в байткод
- Определяет, какие компоненты реально нужны

# Что Quarkus делает на этапе сборки

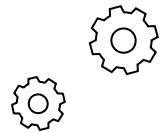
- Читает часть конфигурации приложения
- Сканирует
- Стр
- Ген вмешательство в байткод
- Определяет, какие компоненты реально нужны

Всё, что можно вычислить заранее — **вычисляется заранее**

# Почему это важно

- Неиспользуемые компоненты **удаляются**
- Исключается лишний код и зависимости
- Генерируются точные инструкции запуска
- Приложение стартует почти без прогрева и быстрый scale up




# Build Time

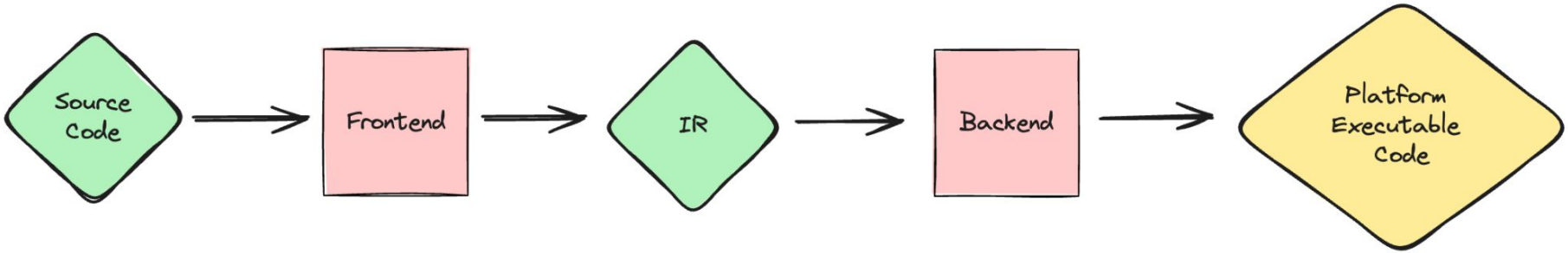



```
public class QuarkusCompiler
```

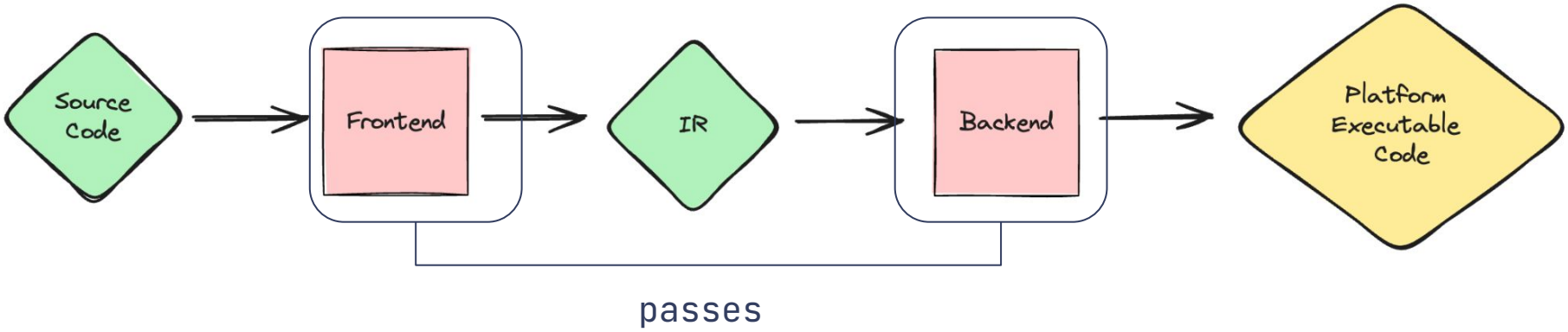



# Значит, надо немного погрузиться в компиляторный мир

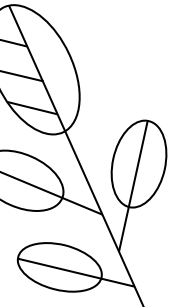
# Типичный процессинг компилятора



# Типичный процессинг компилятора

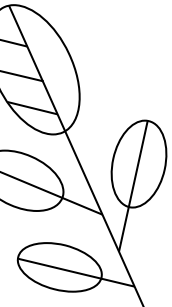


# Небольшой квиз



# Лексика, семантика, синтаксис

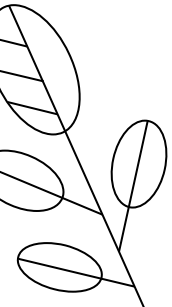
```
fun fo@o() {
    return listOf("Error")
}
```



# Лексика, семантика, синтаксис

Лексическая ошибка

```
fun fo@o() {
    return listOf("Error")
}
```

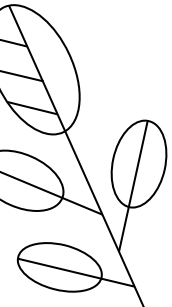


# Лексика, семантика, синтаксис

Лексическая ошибка

```
fun fo@o() {
    return listOf("Error")
}
```

```
fun foo() {
    return listOf("Error")
}
```



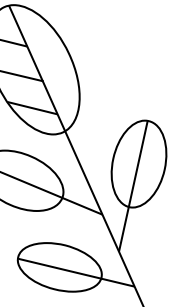
# Лексика, семантика, синтаксис

Лексическая ошибка

```
fun fo@o() {
    return listOf("Error")
}
```

Семантическая ошибка

```
fun foo() {
    return listOf("Error")
}
```



# Лексика, семантика, синтаксис

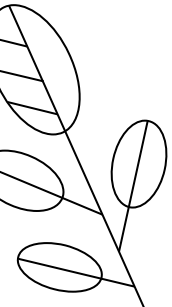
Лексическая ошибка

```
fun fo@o() {
    return listOf("Error")
}
```

Семантическая ошибка

```
fun foo() {
    return listOf("Error")
}
```

```
fun foo() { (.)(.)
    return listOf("Error")
}
```



# Лексика, семантика, синтаксис

Лексическая ошибка

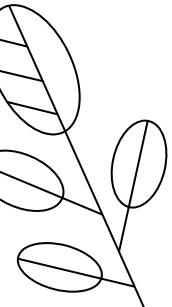
```
fun fo@o() {
    return listOf("Error")
}
```

Семантическая ошибка

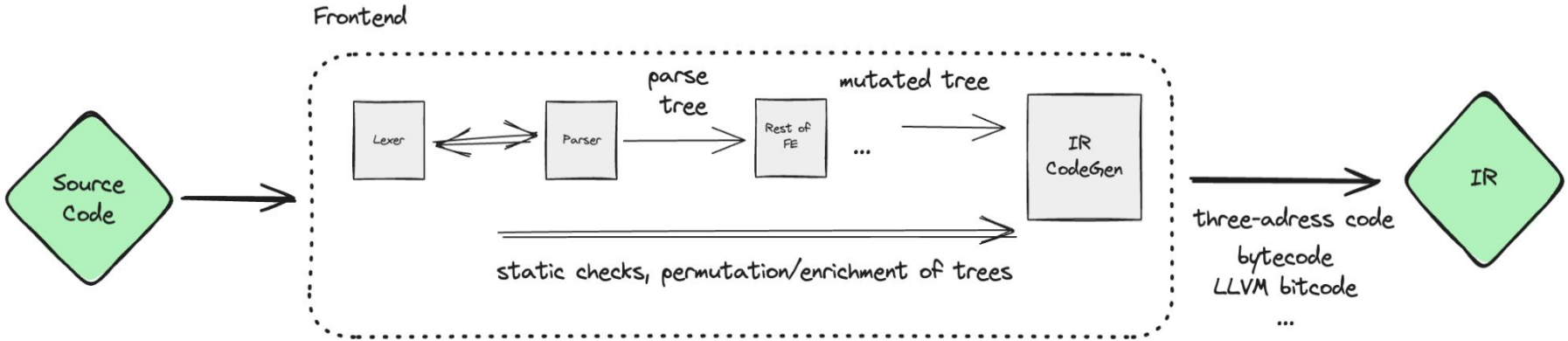
```
fun foo() {
    return listOf("Error")
}
```

Синтаксическая ошибка

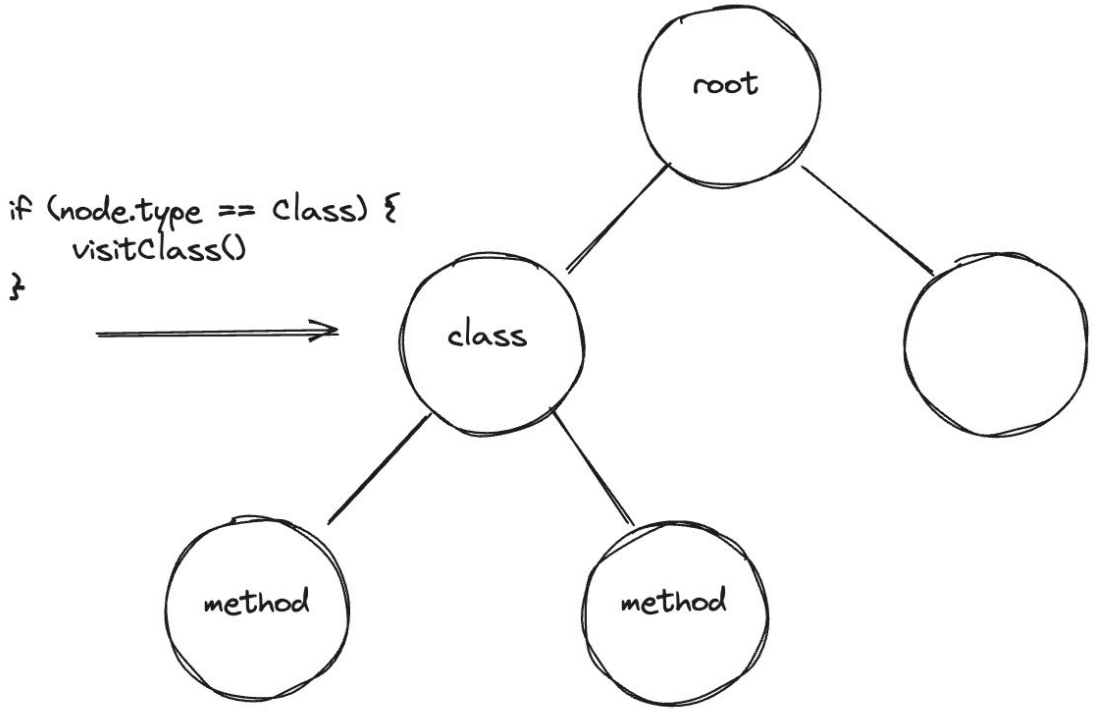
```
fun foo() { (.)(.)
    return listOf("Error")
}
```



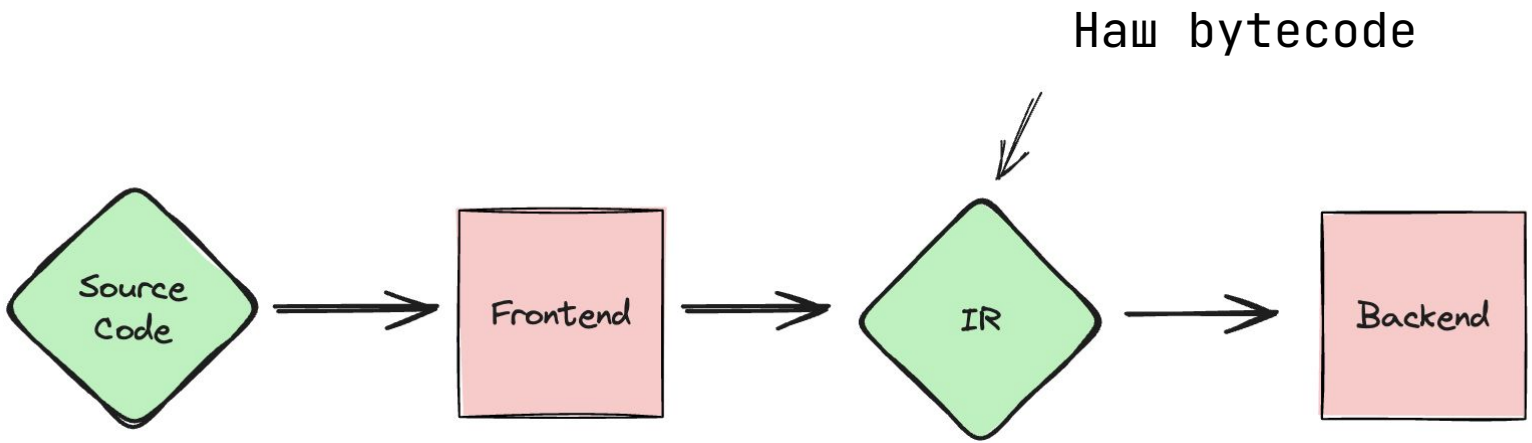
# Типичный процессинг компилятора



# Вмешательство в код



# Можно вмешиваться во фронтенд, а можно в IR



# Вмешаться в bytecode!



ASM



Кем используется?

- the OpenJDK, to generate the lambda call sites, and also in the Nashorn compiler;
- the Groovy compiler and the Kotlin compiler;
- Cobertura and Jacoco, to instrument classes in order to measure code coverage;
- Byte Buddy, to dynamically generate classes, itself used in other projects such as Mockito (to generate mock classes);
- Gradle, to generate some classes at runtime.

# Вмешаться в bytecode!



ASM



Кем используется?

- ~~the OpenJDK, to generate the lambda call sites, and also in the Nashorn compiler;~~
- the Groovy compiler and the Kotlin compiler;
- Cobertura and Jacoco, to instrument classes in order to measure code coverage;
- Byte Buddy, to dynamically generate classes, itself used in other projects such as Mockito (to generate mock classes);
- Gradle, to generate some classes at runtime.

# Вмешаться в bytecode!



ASM



Кем используется?

**JEP 484: Class-File API**

- ~~the OpenJDK, to generate the lambda call sites, and also in the Nashorn compiler;~~
- the Groovy compiler and the Kotlin compiler;
- Cobertura and Jacoco, to instrument classes in order to measure code coverage;
- Byte Buddy, to dynamically generate classes, itself used in other projects such as Mockito (to generate mock classes);
- Gradle, to generate some classes at runtime.

# Вмешаться в bytecode



ASM



**JEP 484: Class-File API**

Кем используется?

- ~~the OpenJDK, to generate t~~
- the
- Cober
- Byte
  (to g
- Gradl

as Mockito

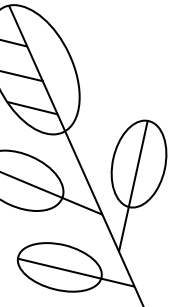
**Goals**

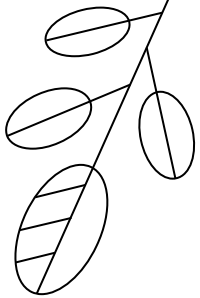
- Provide an API for processing class files that tracks the `class` file format defined by the Java Virtual Machine Specification.
- Enable JDK components to migrate to the standard API, and eventually remove the JDK's internal copy of the third-party ASM library.



ASM

```
class Example {
    int keepMe;
    String removeMe;
}
```





```java
import org.objectweb.asm.ClassVisitor;
import org.objectweb.asm.FieldVisitor;
import org.objectweb.asm.Opcodes;

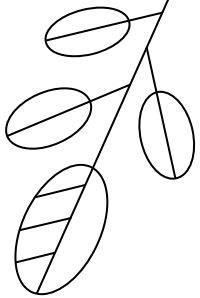
public class RemoveFieldClassVisitor extends ClassVisitor {

    public RemoveFieldClassVisitor(ClassVisitor cv) {
        super(Opcodes.ASM9, cv);
    }

    @Override
    public FieldVisitor visitField(
        int access,
        String name,
        String descriptor,
        String signature,
        Object value
    ) {
        if (fieldName.equals("removeMe")) {
            return null;
        }

        return super.visitField(access, name, descriptor, signature, value);
    }
}
```







```java
import org.objectweb.asm.ClassReader;
import org.objectweb.asm.ClassWriter;

import java.io.IOException;

public class Transformer {

   public static byte[] transform(byte[] originalClass) {
       ClassReader reader = new ClassReader(originalClass);
       ClassWriter writer = new ClassWriter(reader, 0);

       RemoveFieldClassVisitor visitor =
           new RemoveFieldClassVisitor(writer);

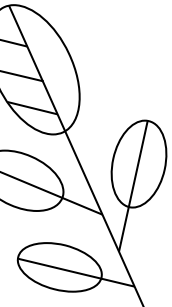
       reader.accept(visitor, 0);

       return writer.toByteArray();
   }
}
```

В итоге логику “тяжёлой работы” можно перенести таким простым упражнением на build-time

Какие оптимизации выполняются на этапе сборки:

- Сканирование classpath
- Загрузка и валидация конфигурации
- Подготовка dependency injection

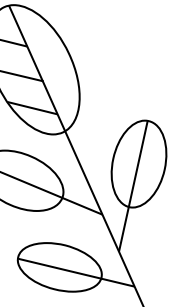


# ClassTransformingBuildStep

```
ClassReader cr = new ClassReader(classData);
ClassWriter writer = new QuarkusClassWriter(cr, ...);

ClassVisitor visitor = writer;
for (BiFunction<String, ClassVisitor, ClassVisitor> i : visitors) {
   visitor = i.apply(className, visitor);
}

cr.accept(visitor, classReaderOptions);
data = writer.toByteArray();
```



# ClassTransformingBuildStep: зачем?

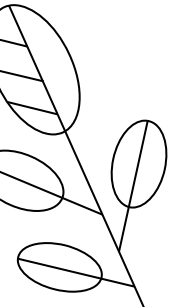
Hibernate, например

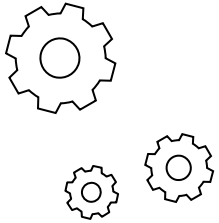


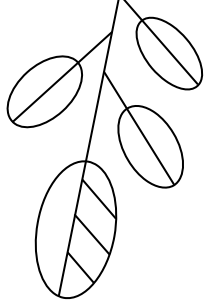
# CDI

jakarta.enterprise.cdi

**Arc**





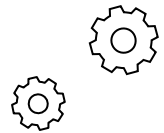


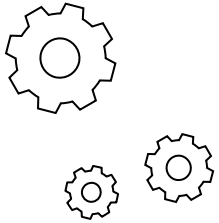
# Азы и терминология

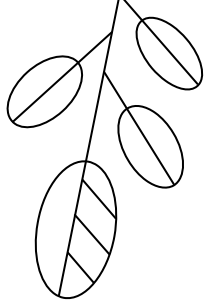
**Arc** - приблизительная имплементация спецификации Jakarta Contexts and Dependency Injection 4.1

```
quarkus.arc.strict-compatibility=true
```





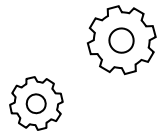




**Arc** - приблизительная имплементация спецификации [Jakarta Contexts and Dependency Injection 4.1]

Работает за счет **ASM**, библиотеки для манипуляций с байткодом





# Contexts and Dependency Injection

```
@ApplicationScoped
public class UserService {
    public User findById(Long id) {
        return new User(id, "John");
    }
}
```

Singleton, с дополнительными сгенерированными обертками и прокси

# Contexts and Dependency Injection

```java
@ApplicationScoped
public class OrderService {

    // ===== Field или setter
    @Inject
    UserService userService;



}
```

# Contexts and Dependency Injection

```java
@ApplicationScoped
public class OrderService {

    // ===== Field или setter
    @Inject
    UserService userService;

    // ===== Конструктор
    private final PaymentService paymentService;

    @Inject
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }



}
```

# Contexts and Dependency Injection

```java
@ApplicationScoped
public class OrderService {

    // ===== Field или setter
    @Inject
    UserService userService;

    // ===== Конструктор
    private final PaymentService paymentService;

    @Inject
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

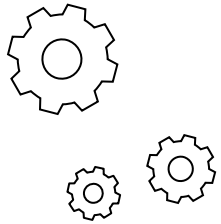
    // ===== Метод
    @Inject
    void configure(ConfigService config, LogService log) {
        // Вызывается после создания объекта
    }
}
```

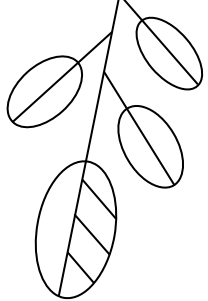
# Contexts and Dependency Injection

```
@RequestScoped
public class RequestContext {
   private String requestId =
           UUID.randomUUID().toString();

   public String getRequestId() {
       return requestId;
   }
}
```

Новый экземпляр, например для каждого HTTP запроса

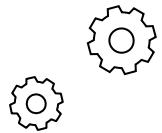




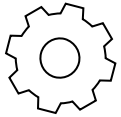
`@Dependent` - создается новый экземпляр каждый раз при инъекции

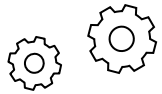
`@Singleton` - похож на `@ApplicationScoped`, но не создается proxy

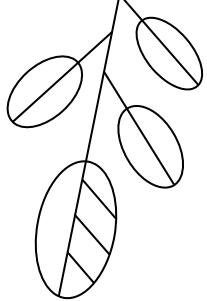




# ARC / Jakarta **Interceptors**







@Transactional

@CircuitBreaker

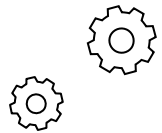
@RateLimit

@Timeout

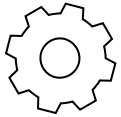
@Retry

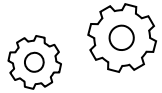
@Fallback

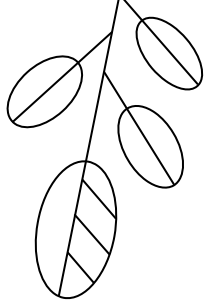




# ARC / Jakarta Interceptors







@Transactional

Arc во время **build-time**:

@CircuitBreaker

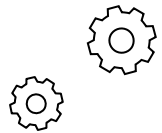
@RateLimit

@Timeout

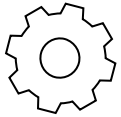
@Retry

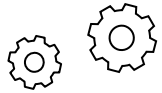
@Fallback

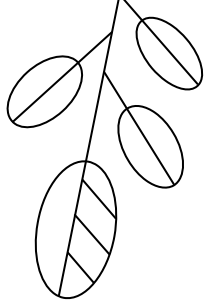




# ARC / Jakarta Interceptors







@Transactional

@CircuitBreaker

@RateLimit

@Timeout

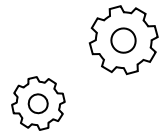
@Retry

@Fallback

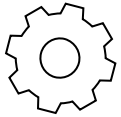
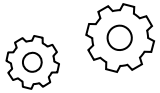
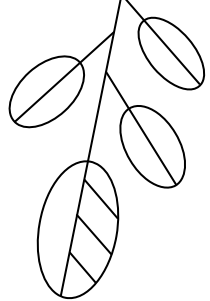
Arc во время build-time:

- Находит всё, что имеет аннотации-интерсепторы итд





# ARC / Jakarta Interceptors

@Transactional

@CircuitBreaker

@RateLimit

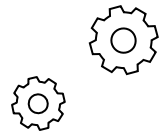
@Timeout

@Retry

@Fallback

Arc во время build-time:

- Находит всё, что имеет аннотации-интерсепторы итд
- Генерирует байткод, который не вызывает метод напрямую, а вставляет вызов цепочки интерсепторов

# ARC / Jakarta Interceptors

@Transactional

@CircuitBreaker

@RateLimit

@Timeout

@Retry

@Fallback

Arc во время build-time:

- Находит всё, что имеет аннотации-интерсепторы итд
- Генерирует байткод, который не вызывает метод напрямую, а вставляет вызов цепочки интерсепторов
- В runtime, при вызове метода, сначала выполняются интерсепторы, потом и сам метод

# ARC / Jakarta Interceptors

@Transactional

@RateLimit

@CircuitBreaker

@Timeout

@InterceptorBinding

@RegisterInterceptor

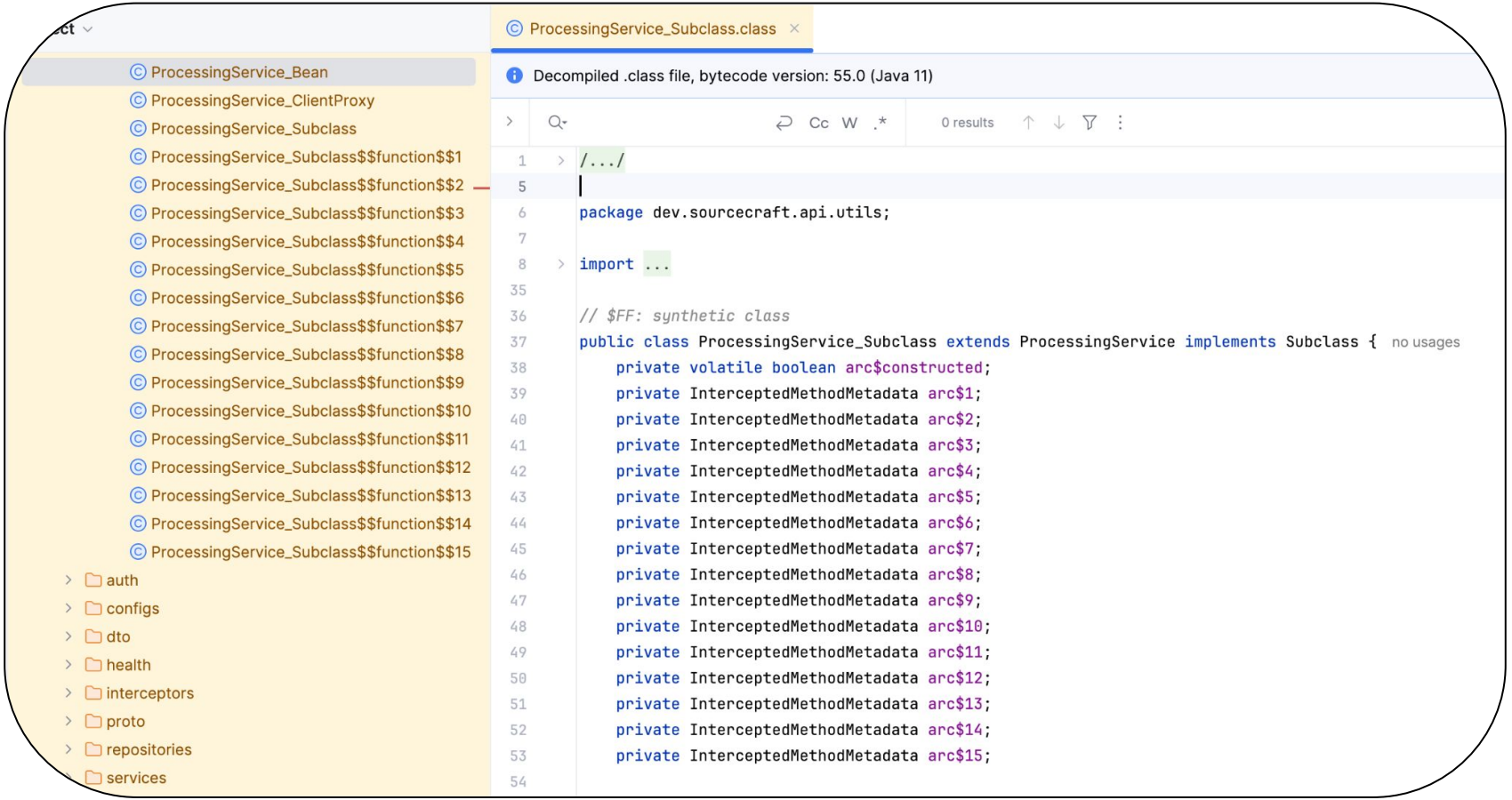
@Fallback

@Retry

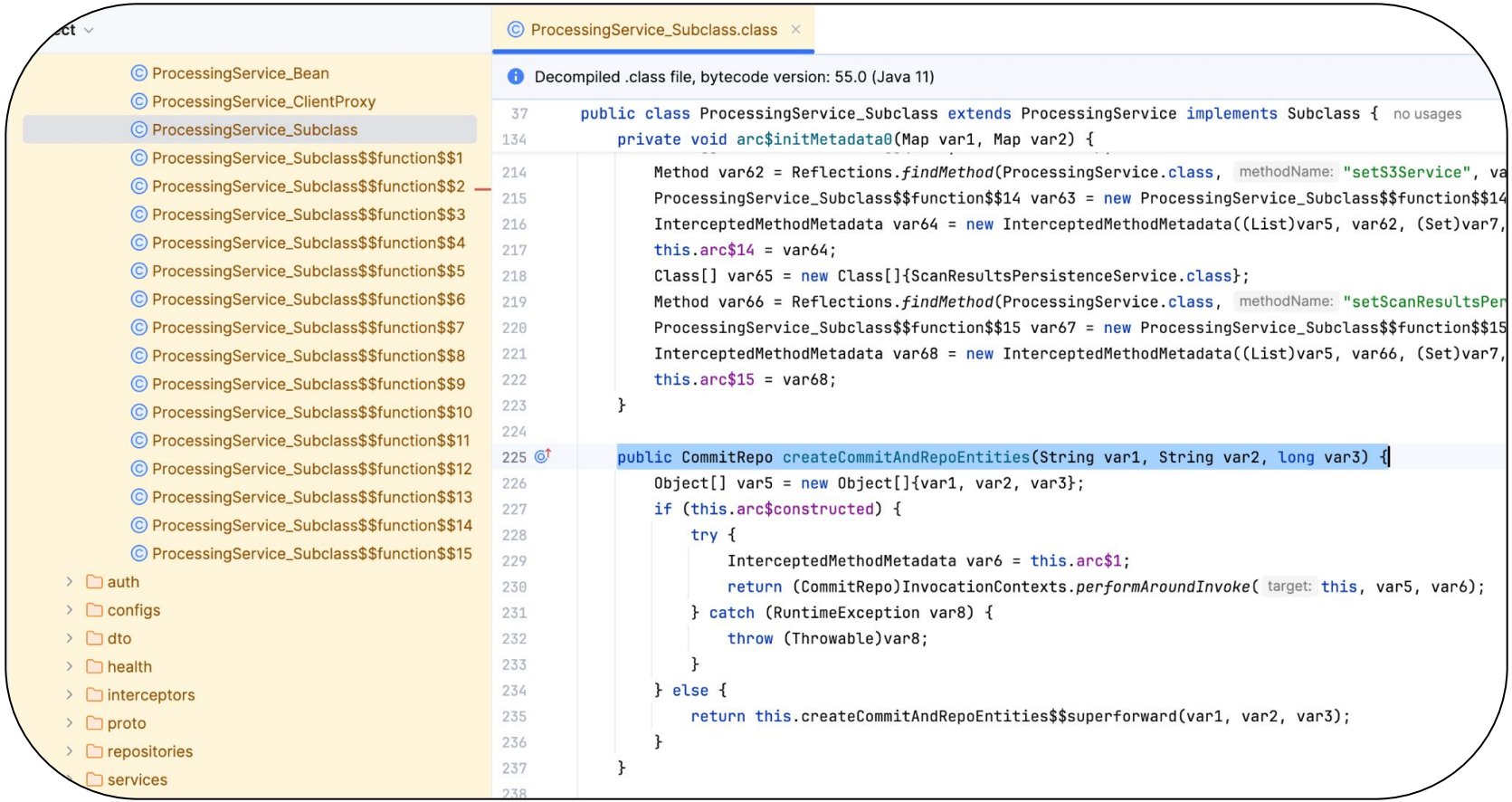
# Bytecode-gen

```
@ApplicationScoped
class ProcessingService {
    fun createCommitAndRepoEntities() {
                ...
     }
}
```

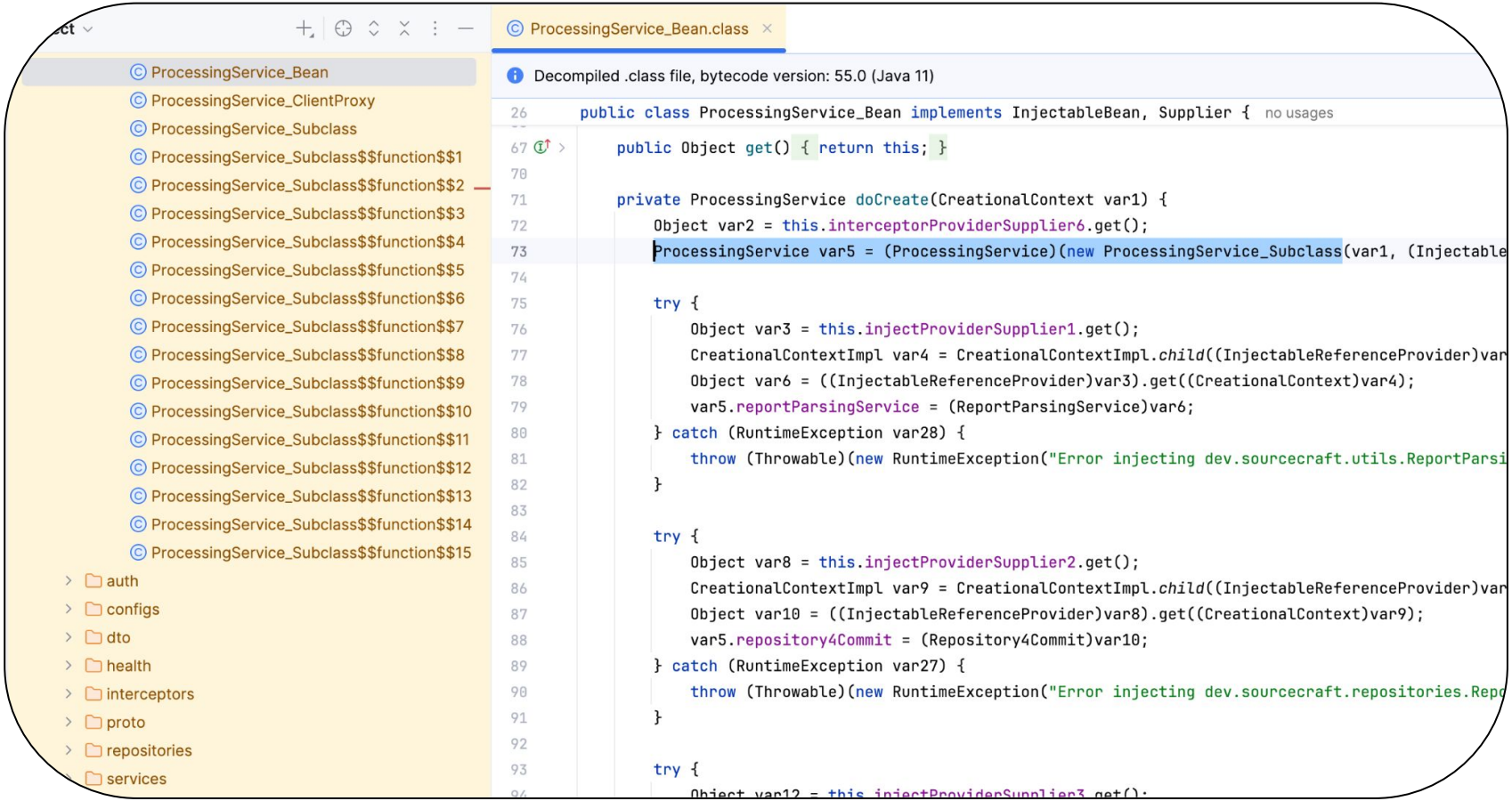
# Subclass - реальный объект



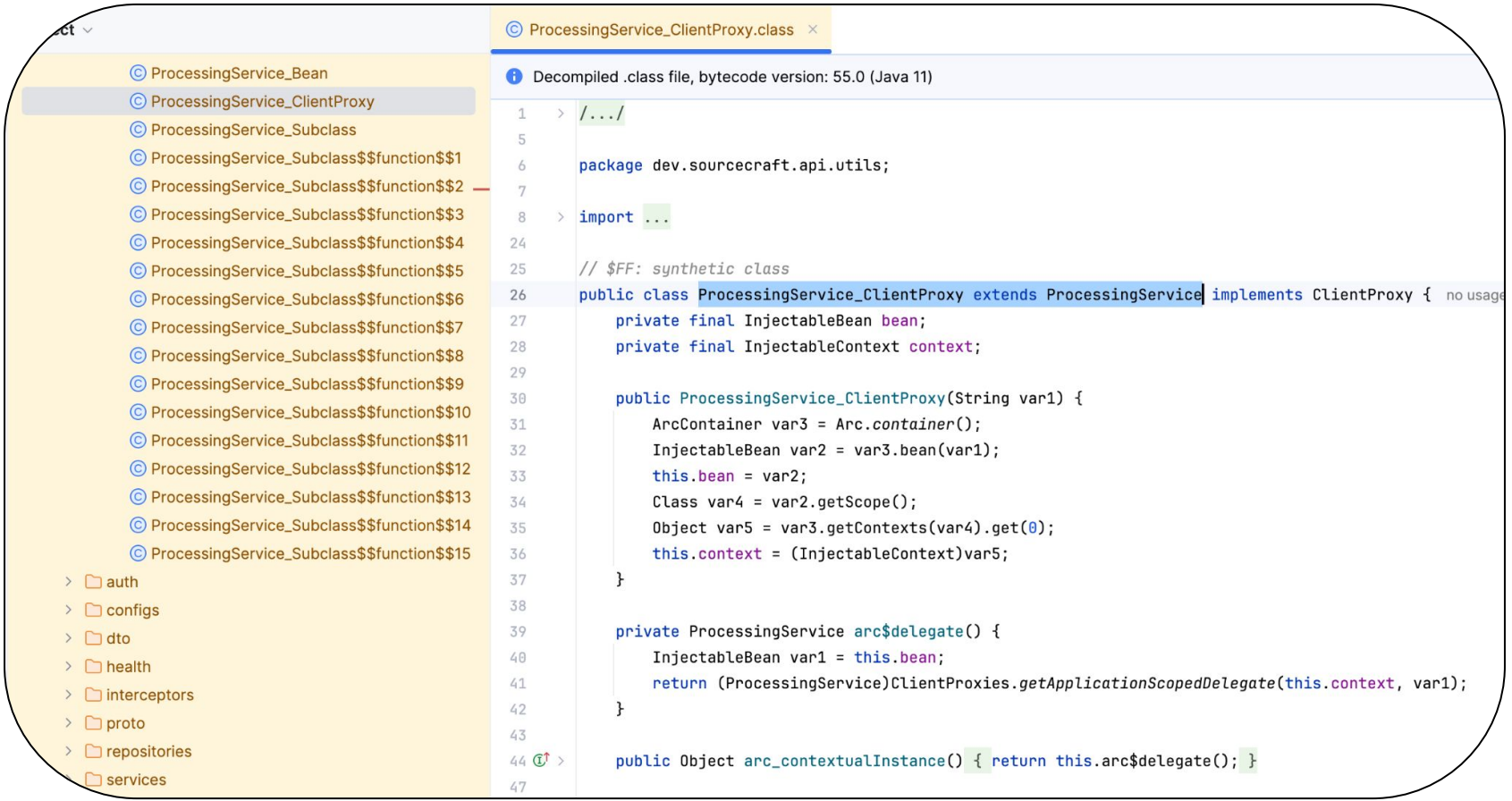
# Subclass - интерсепторы в логике

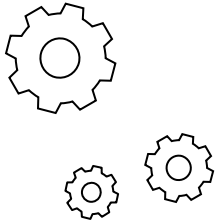
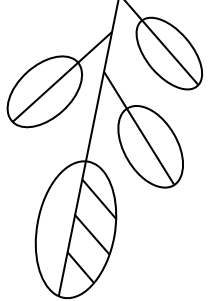


# Bean-конструктор

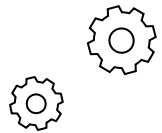


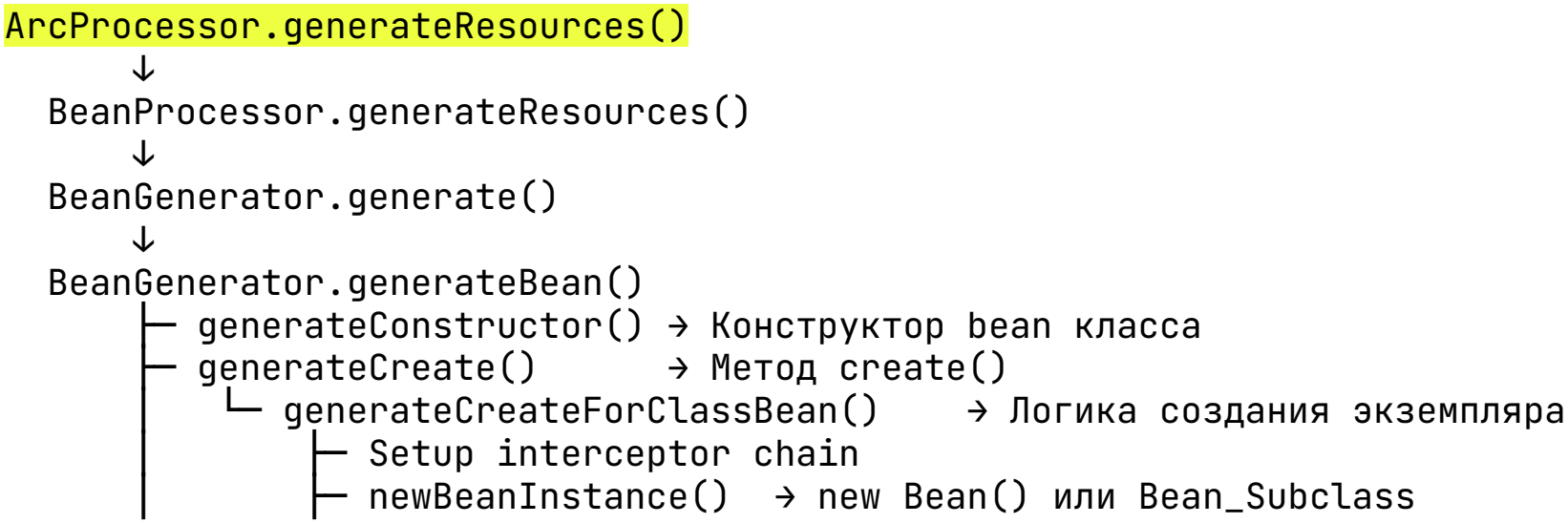
# Client-proxy



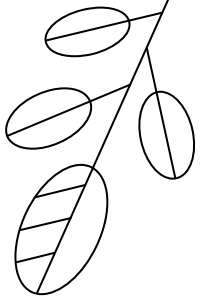



# @Unremovable


ArcProcessor.generateResources()
↓
BeanProcessor.generateResources()
↓
BeanGenerator.generate()
↓
BeanGenerator.generateBean()
├─ generateConstructor() → Конструктор bean класса
├─ generateCreate() → Метод create()
│ └─ generateCreateForClassBean() → Логика создания экземпляра
│ ├─ Setup interceptor chain
│ ├─ newBeanInstance() → new Bean() или Bean_Subclass

```
ArcProcessor.generateResources()
      ↓
  BeanProcessor.generateResources()
      ↓
  BeanGenerator.generate()
      ↓
  BeanGenerator.generateBean()
      ├─ generateConstructor() → Конструктор bean класса
      ├─ generateCreate()      → Метод create()
      │   └─ generateCreateForClassBean()    → Логика создания экземпляра
      │       ├─ Setup interceptor chain
      │       ├─ newBeanInstance()  → new Bean() или Bean_Subclass
```



## newBeanInstance()



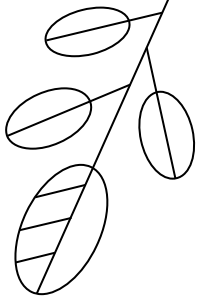
```
List<InterceptorInfo> interceptors = bean.getBoundInterceptors();
List<DecoratorInfo> decorators = bean.getBoundDecorators();

// 1. constructor injection points
for (int i = 0; i < injectionPoints.size(); i++) {
   paramTypes.add(classDescOf(injectionPoints.get(i).getType()));
   args.add(providers.get(i));
}

// 2. ctx
paramTypes.add(Reflection2Gizmo.classDescOf(CreationalContext.class));
args.add(parentCC);

// 3. interceptors (wrapped if needed)
for (int i = 0; i < interceptors.size(); i++) {
   paramTypes.add(Reflection2Gizmo.classDescOf(InjectableInterceptor.class));
   args.add(providers.get(injectionPoints.size() + i));
}

// 4. decorators
for (int i = 0; i < decorators.size(); i++) {
   paramTypes.add(Reflection2Gizmo.classDescOf(InjectableDecorator.class));
   args.add(providers.get(injectionPoints.size() + interceptors.size() + i));
}
```



# newBeanInstance()



```
List<InterceptorInfo> interceptors = bean.getBoundInterceptors();
List<DecoratorInfo> decorators = bean.getBoundDecorators();

// 1. constructor injection points
for (int i = 0; i < injectionPoints.size(); i++) {
   paramTypes.add(classDescOf(injectionPoints.get(i).getType()));
   args.add(providers.get(i));
}

// 2. ctx
paramTypes.add(Reflection2Gizmo.classDescOf(CreationalContext.class));
args.add(parentCC);

// 3. interceptors (wrapped if needed)
for (int i = 0; i < interceptors.size(); i++) {
   paramTypes.add(Reflection2Gizmo.classDescOf(InjectableInterceptor.class));
   args.add(providers.get(injectionPoints.size() + i));
}

// 4. decorators
for (int i = 0; i < decorators.size(); i++) {
   paramTypes.add(Reflection2Gizmo.classDescOf(InjectableDecorator.class));
   args.add(providers.get(injectionPoints.size() + interceptors.size() + i));
}
```

# Gizmo 1 vs Gizmo 2

First off, Gizmo 1 is based on ASM and Gizmo 2 is based on the ClassFile API (not the one present in the JDK since version 24, but the fork maintained by David Lloyd, which supports Java 17). The ClassFile API itself is very different to ASM, and since the ClassFile API structure guided the Gizmo 2 API structure, that is also very different.



# Gizmo 1 vs Gizmo 2

First off, Gizmo 1 is based on ASM and Gizmo 2 is based on the ClassFile API (not the one present in the JDK since version 24, but the fork maintained by David Lloyd, which supports Java 17). The ClassFile API itself is very different to ASM, and since the ClassFile API structure guided the Gizmo 2 API structure, that is also very different.

```
Gizmo gizmo = Gizmo.create(ClassOutput.fileWriter(Path.of("target")));
gizmo.class_("com.example.Hello", cc → {
    cc.defaultConstructor();

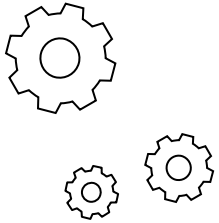
    cc.staticMethod("main", mc → {
        ParamVar args = mc.parameter("args", String[].class);
        mc.body(bc → {
            bc.printf("Hello, World!%n");
            bc.return_();
        });
    });
});
```

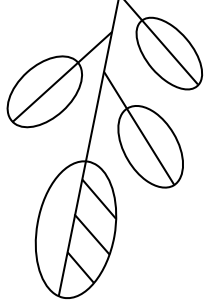


# META-INF/jandex.idx

# META-INF/jandex.idx

.class файлы
↓
Jandex читает байткод
↓
Строит индекс:
классы
методы
поля
аннотации
↓
META-INF/jandex.idx





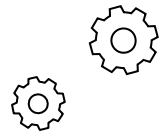
# Где еще пытаются подобным образом достигнуть оптимизации?

Spring AOT

Java AOT (aot cache)

GraalVM (действительно AOT)





483: Ahead-of-Time Class Loading & Linking

**Spring** PetClinic (21,000 classes):

Startup time reduced from

4.48s → 2.60s (42% improvement) Java 24

4.48s → 3.00s (33% improvement) Java 13+, AppCDS

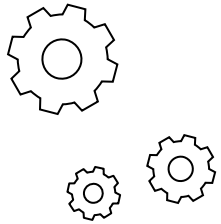
39

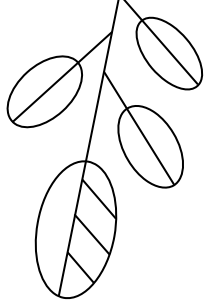


February 26, 2026 #release

# Quarkus 3.32 - Project Leyden integration, more graceful shutdown, automatic Consul registration and more!

By  Guillaume Smet





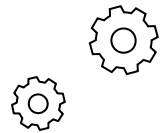
```
./gradle build -Dquarkus.package.jar.aot.enabled=true

./gradlew build quarkusIntTest buildAotEnhancedImage
-Dquarkus.package.jar.aot.enabled=true
```

1. Quarkus builds the application
2. Integration tests (i.e. tests that are annotated with `@QuarkusIntegrationTest`) run against the application with AOT training enabled
3. The JVM captures profiling data during the test run
4. An optimized `app.aot` cache is generated



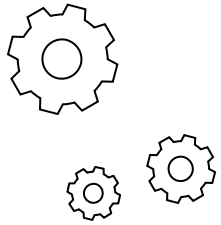
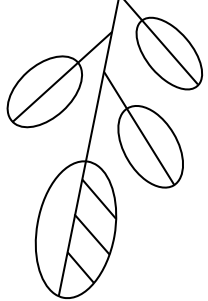
https://quarkus.io/guides/aot



# Выводы

# Выводы

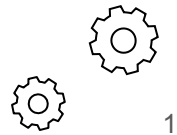
- Выиграть в производительности можно, жертвуя **“универсальностью”** и **“совместимостью”**

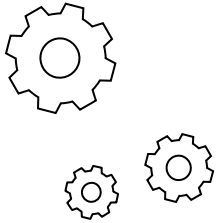



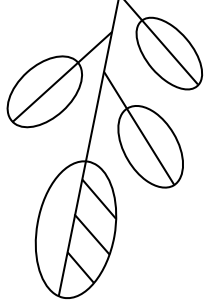
# Выводы

- Выиграть в производительности можно, жертвуя **“универсальностью”** и **“совместимостью”**

- **Берите, что дают:** ограничением ядра фреймворка





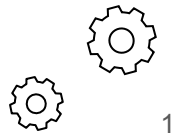
# Выводы

- Выиграть в производительности можно, жертвуя **“универсальностью”** и **“совместимостью”**

- **Берите, что дают:** ограничением ядра фреймворка

- Переносом бОльшей части логики на build time (например, **DI**)







# Спасибо!

Q&A

Мой канал